

Deep Learning

Training Neural Networks

How to **optimize parameters** efficiently?

- Optimizers
- Initialization
- Normalization

Optimizers

Empirical risk minimization

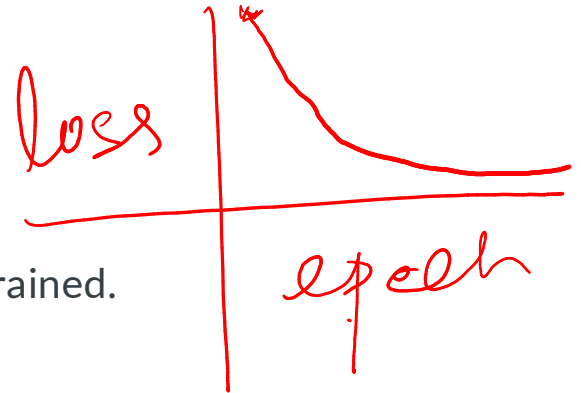
$$\theta_*^{\text{d}} = \arg \min_{\theta} \mathcal{L}(\theta) = \frac{1}{N} \sum_{n=1}^N \ell(y_n, f(\mathbf{x}_n; \theta)).$$

A practical recommendation

Training a massive deep neural network is long, complex and sometimes confusing.

A first step towards understanding, debugging and optimizing neural networks is to make use of visualization tools for

- plotting losses and metrics,
- visualizing computational graphs,
- or showing additional data as the network is being trained.



☒ Show data download links☒ Ignore outliers in chart scalingTooltip sorting
method: default

Smoothing

 0.6

Horizontal Axis

STEP

RELATIVE

WALL

Runs

Write a regex to filter runs

☒ ☐ train☒ ☐ eval

TOGGLE ALL RUNS

/tmp/mnist-logs

Filter tags (regular expressions supported)

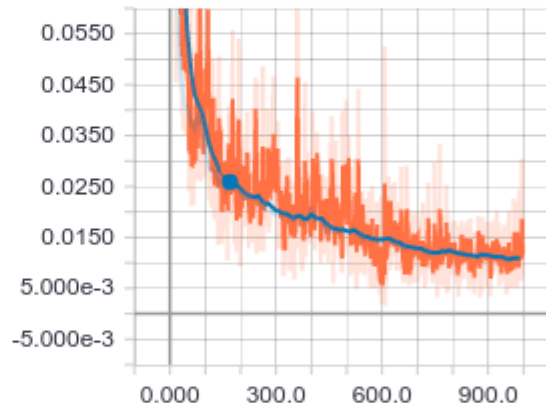
accuracy

1

cross entropy

1

cross entropy



run to download CSV JSON

	Name	Smoothed	Value	Step	Time	Relative
●	eval	0.02591	0.02550	170.0	Mon Sep 12, 15:40:41	8s
●	train	0.02851	0.03362	166.0	Mon Sep 12, 15:40:40	7s

mean

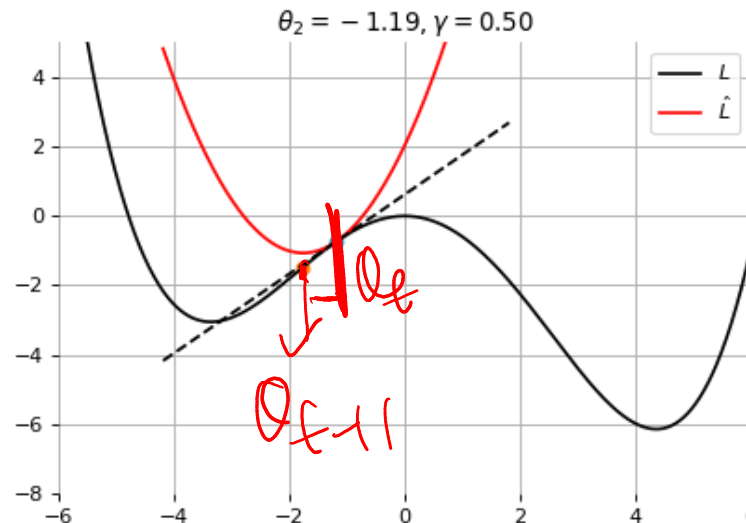
4

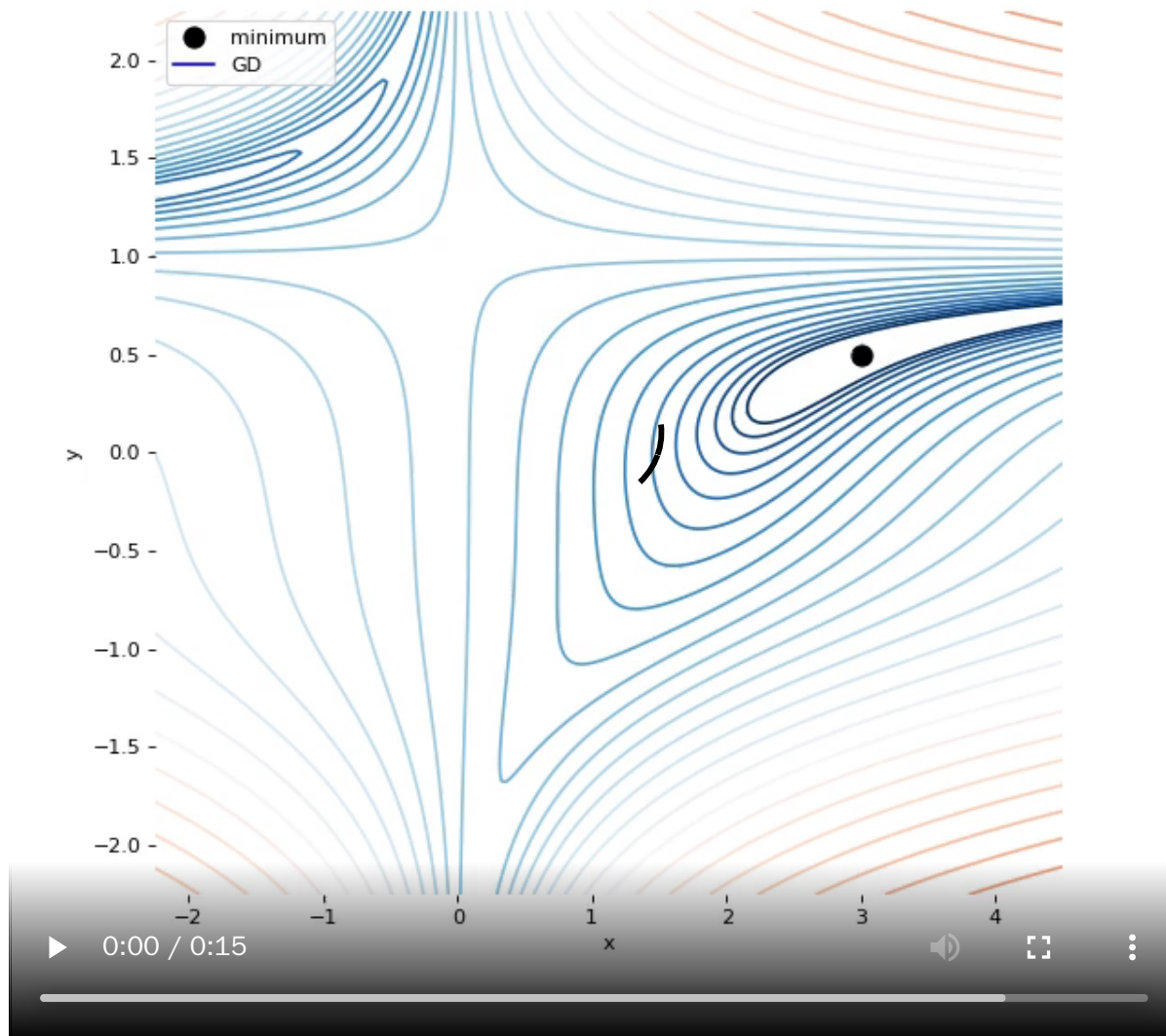
Gradient descent

To minimize $\mathcal{L}(\theta)$, standard batch gradient descent (GD) consists in applying the update rule

$$g_t = \frac{1}{N} \sum_{n=1}^N \nabla_{\theta} \ell(y_n, f(\mathbf{x}_n; \theta_t))$$
$$\theta_{t+1} = \theta_t - \gamma g_t$$

where γ is the learning rate.



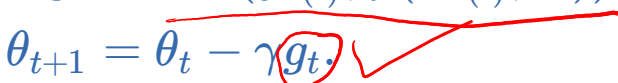


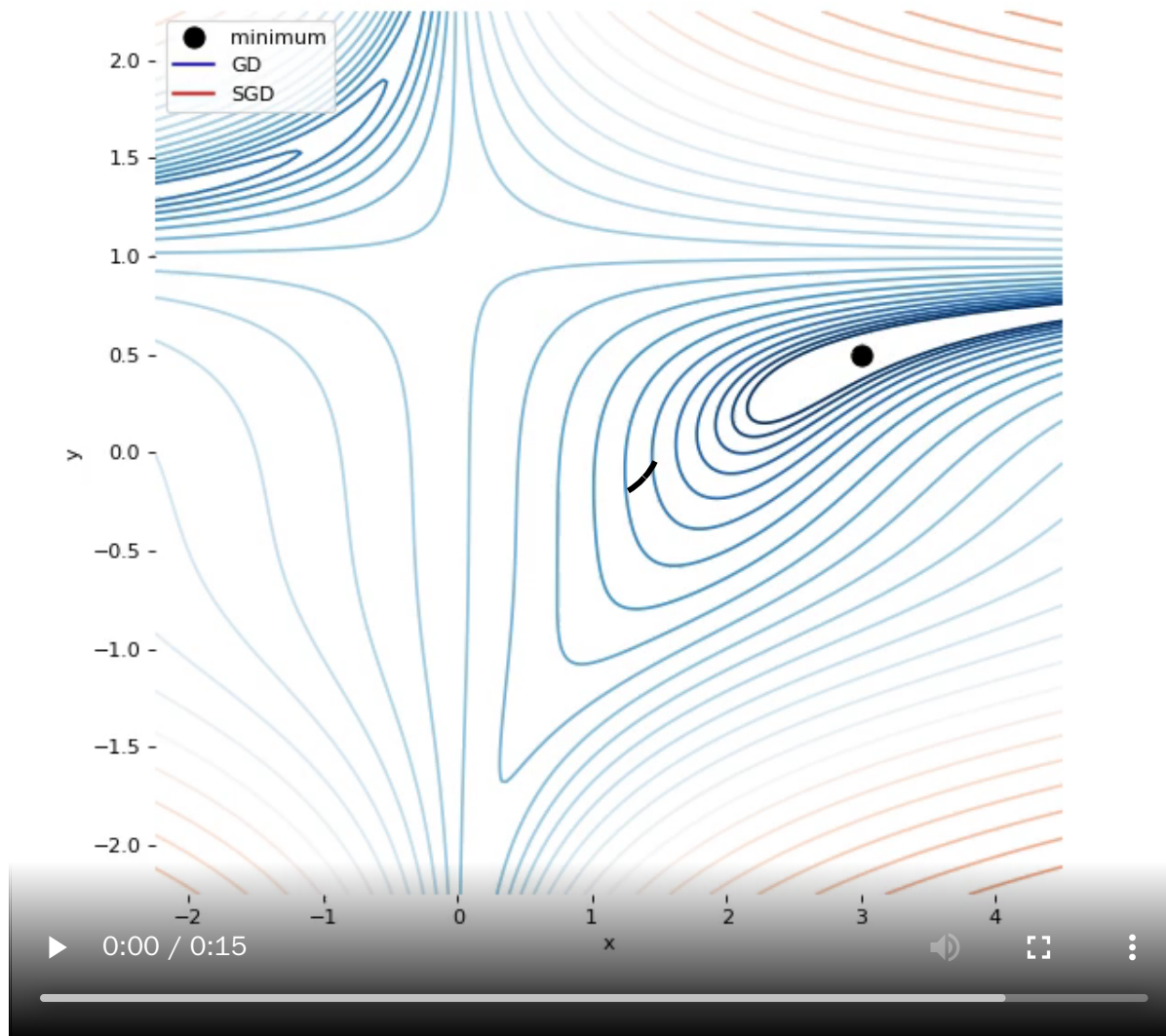
Stochastic gradient descent

While it makes sense to compute the gradient exactly,

- it takes time to compute and becomes inefficient for large N ,
- it is an empirical estimation of an hidden quantity (the expected risk), and any partial sum is also an unbiased estimate, although of greater variance.

To reduce the computational complexity, **stochastic gradient descent** (SGD) consists in updating the parameters after every sample

$$g_t = \nabla_{\theta} \ell(y_{n(t)}, f(\mathbf{x}_{n(t)}; \theta_t))$$
$$\theta_{t+1} = \theta_t - \gamma g_t$$




While being computationally faster than batch gradient descent,

- gradient estimates used by SGD can be **very noisy**, which may help escape from local minima;
- SGD does not benefit from the speed-up of **batch-processing**.

Mini-batching

Instead, **mini-batch** SGD consists in visiting the samples in mini-batches and updating the parameters each time

$$g_t = \frac{1}{B} \sum_{b=1}^B \nabla_{\theta} \ell(y_{n(t,b)}, f(\mathbf{x}_{n(t,b)}; \theta_t))$$
$$\theta_{t+1} = \theta_t - \gamma g_t,$$

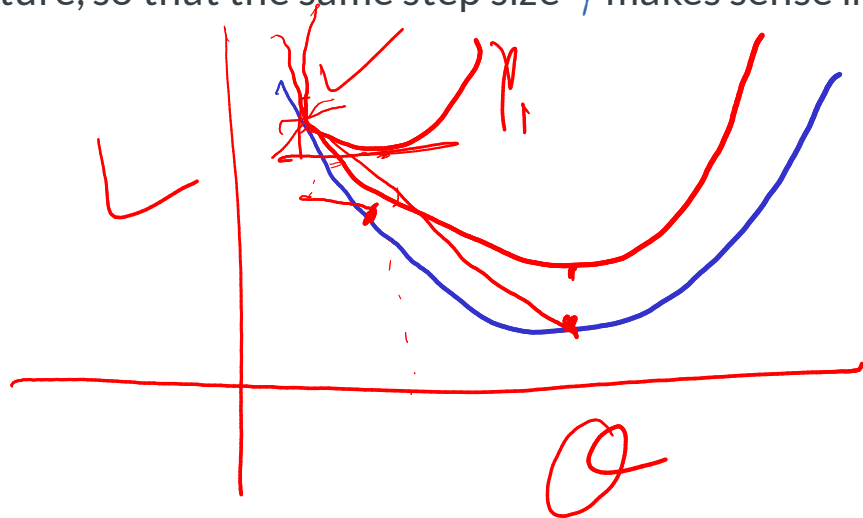
where the order $n(t, b)$ to visit the samples can be either sequential or random.

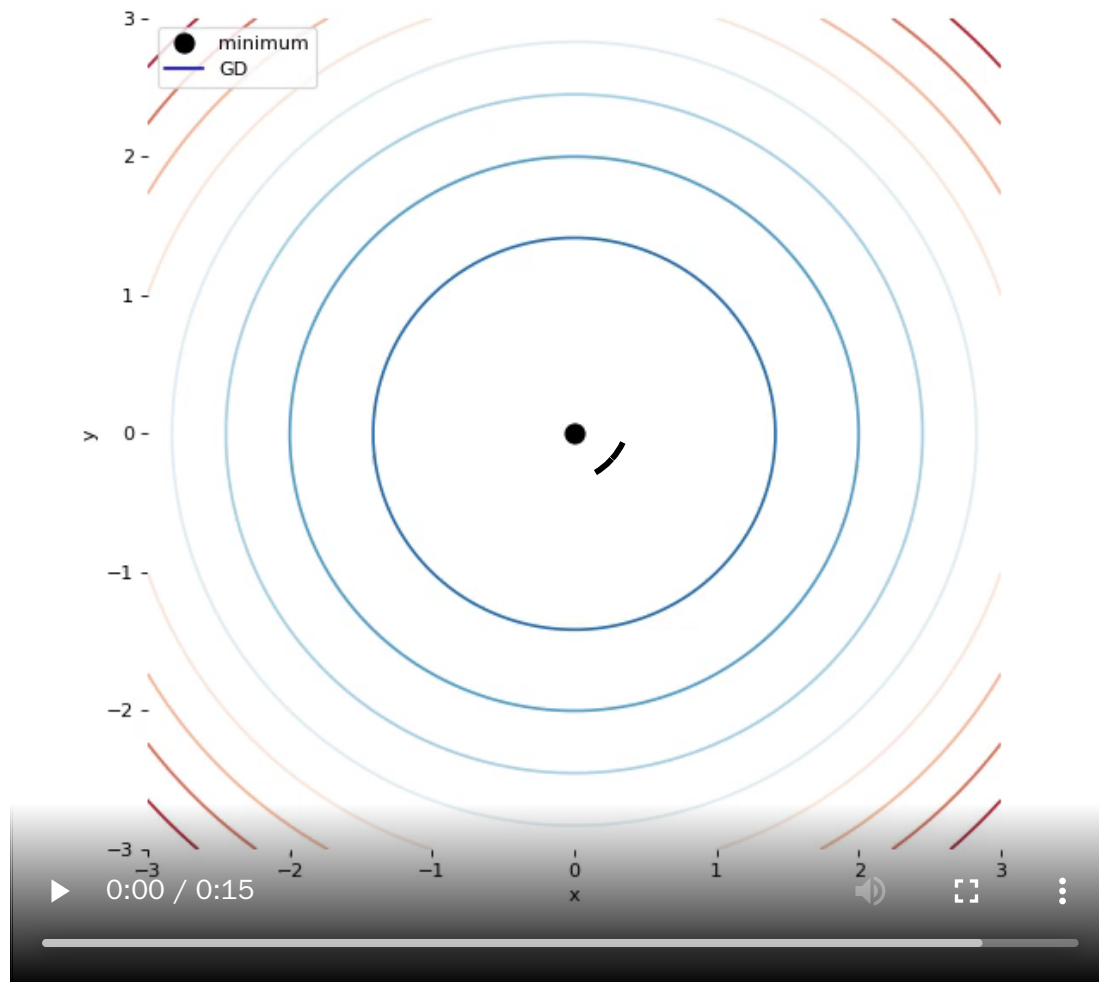
- Increasing the batch size B reduces the variance of the gradient estimates and enables the speed-up of batch processing.
- The interplay between B and γ is still unclear.

Limitations

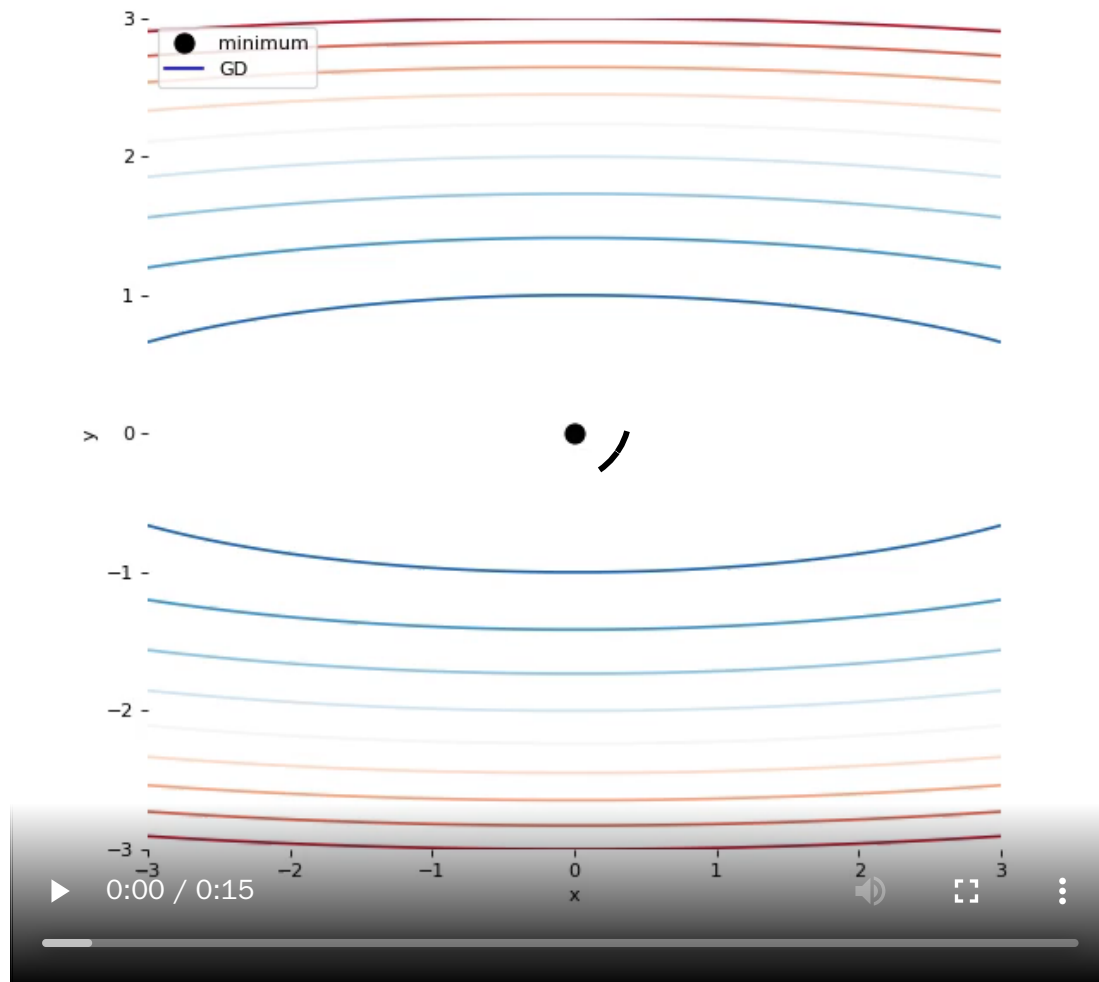
The gradient descent method makes strong assumptions about

- the magnitude of the local curvature to set the step size, ✓
- the isotropy of the curvature, so that the same step size γ makes sense in all directions.

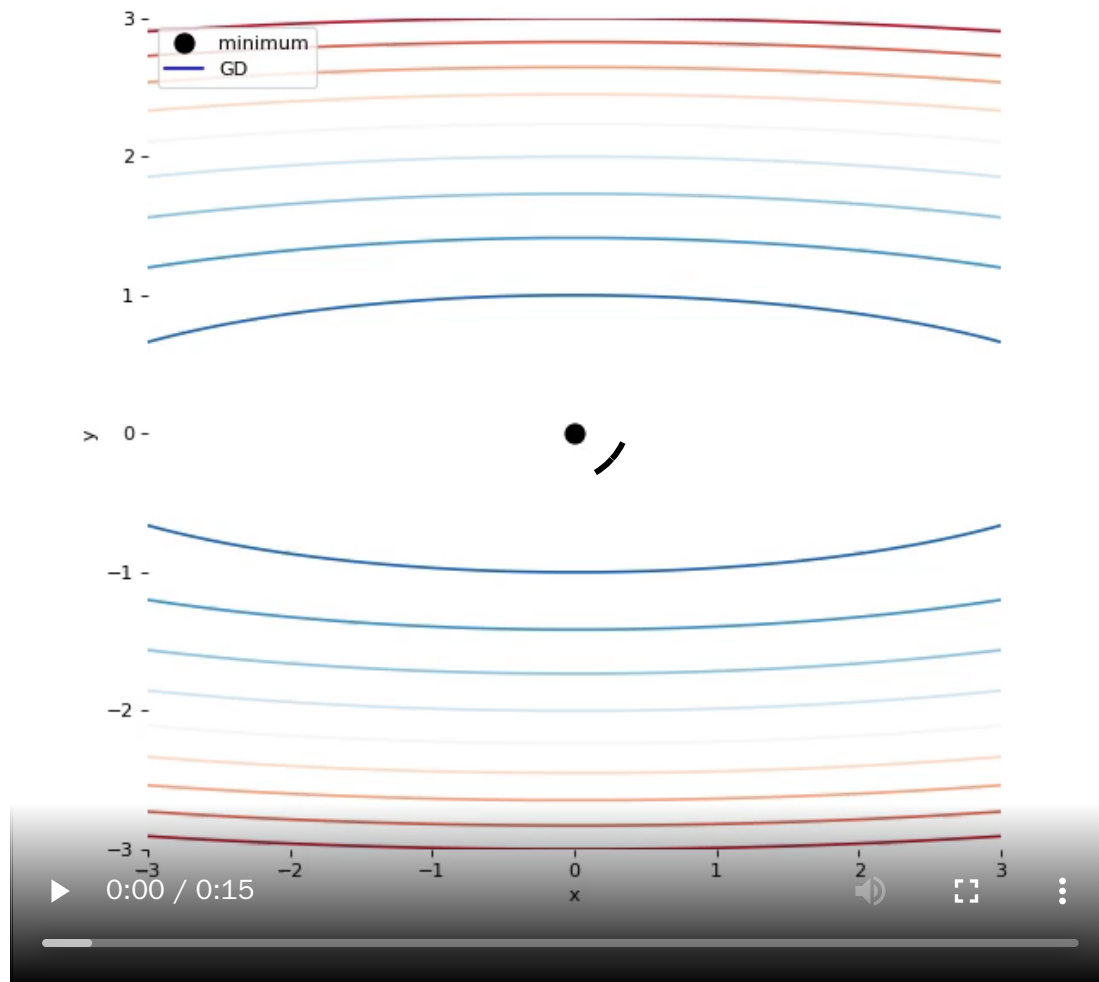




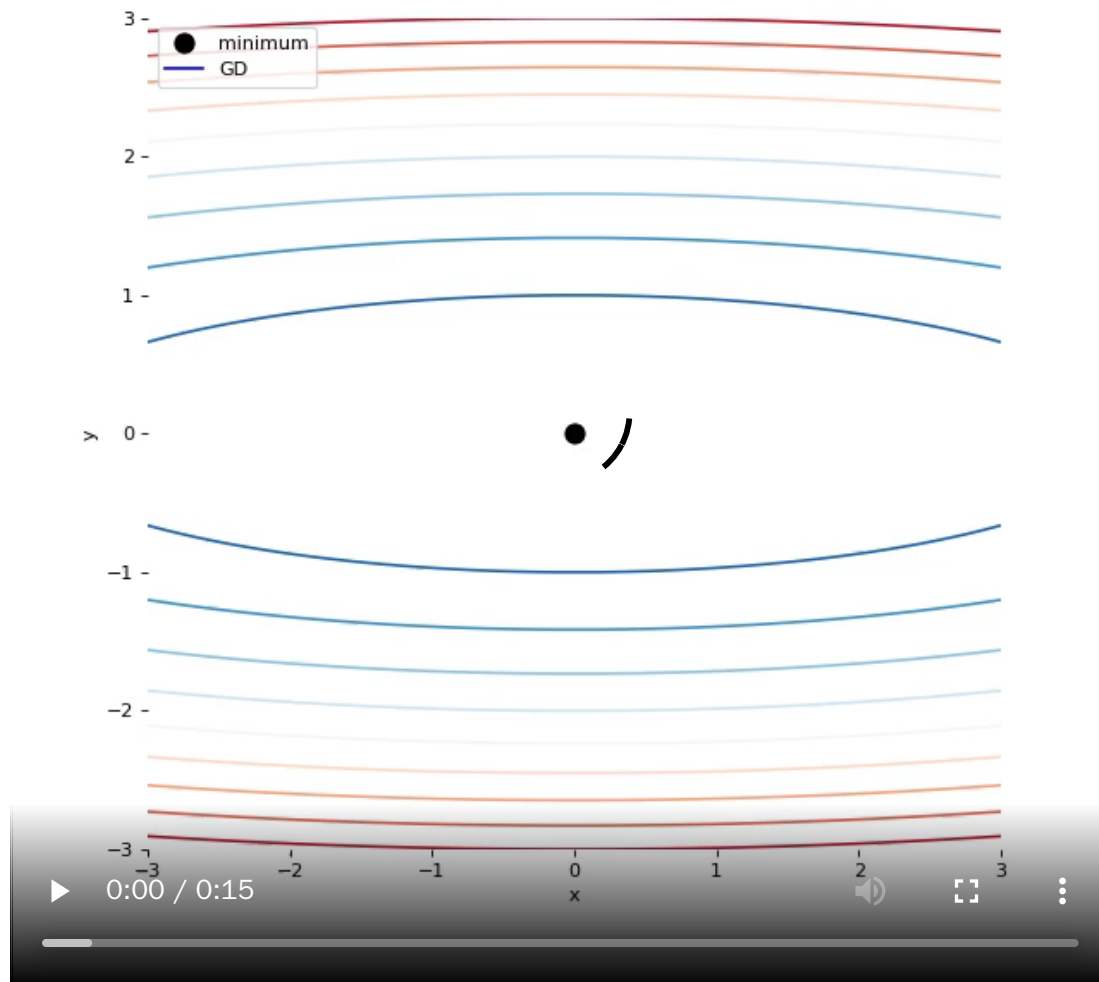
$$\gamma = 0.01$$



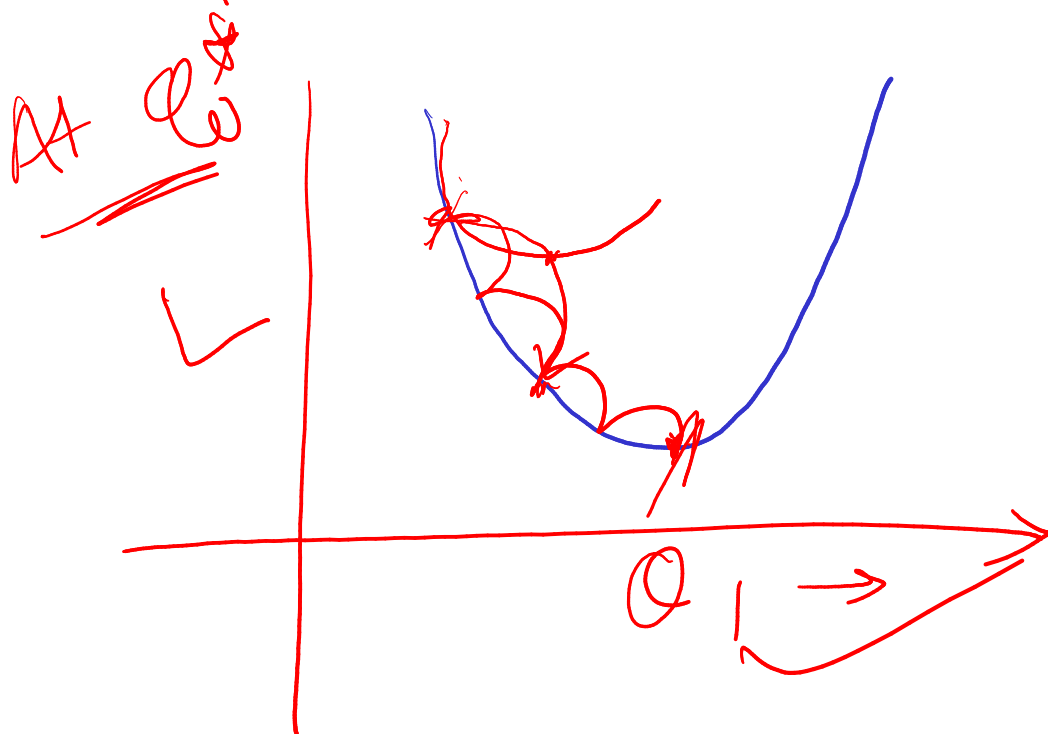
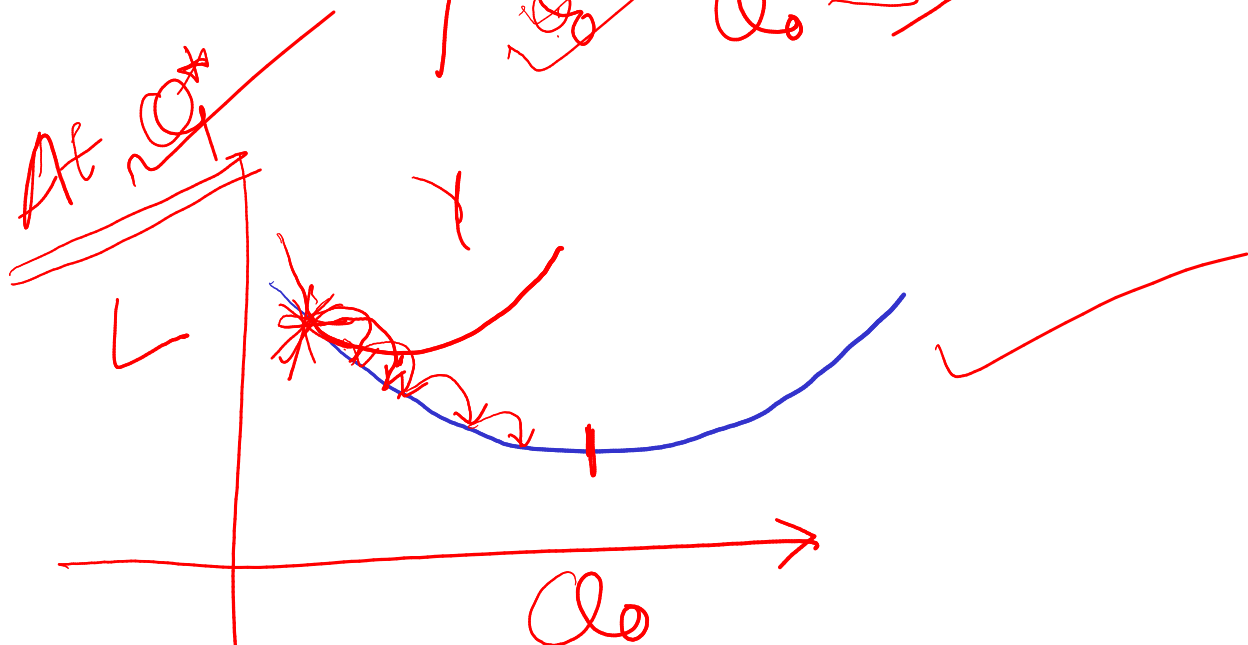
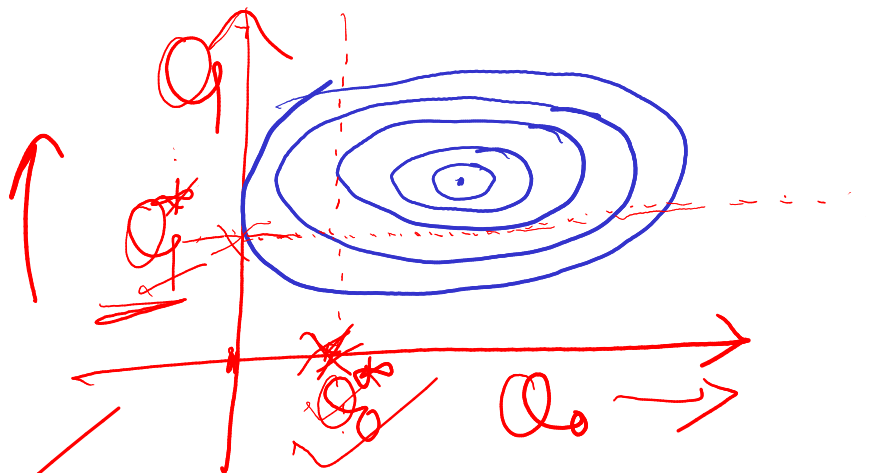
$$\gamma = 0.01$$



$$\gamma = 0.1$$



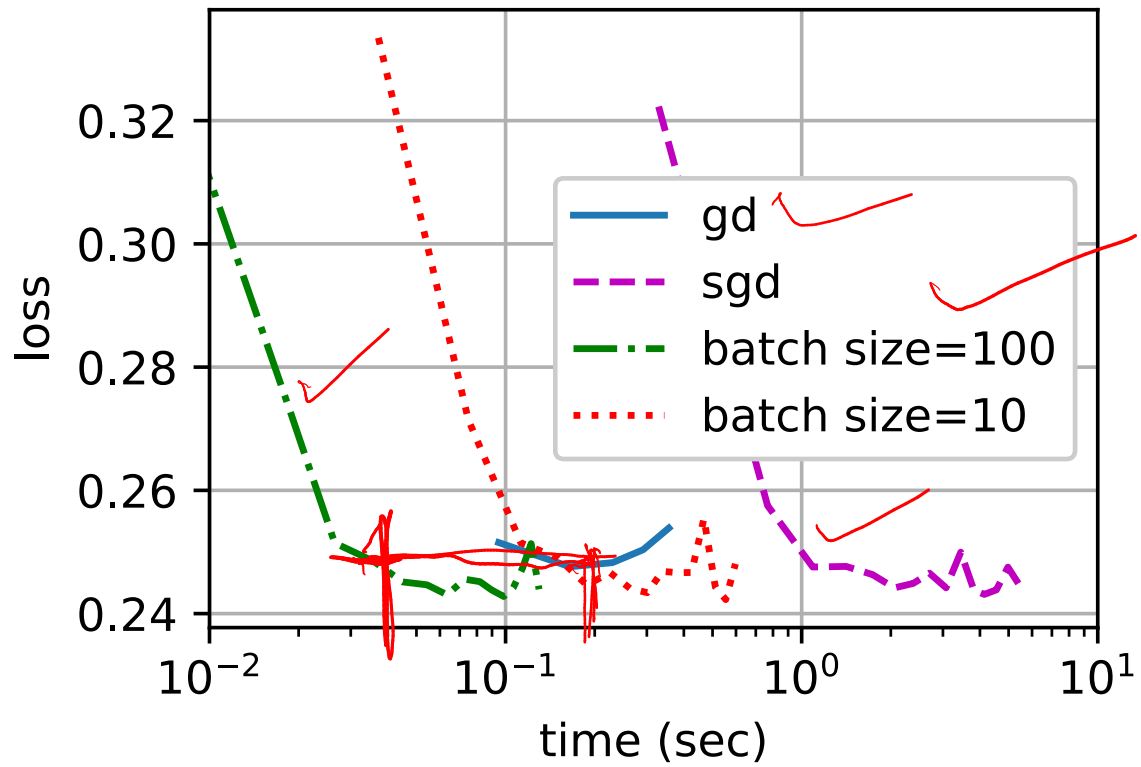
$$\gamma = 0.4$$



The tradeoffs of large-scale learning

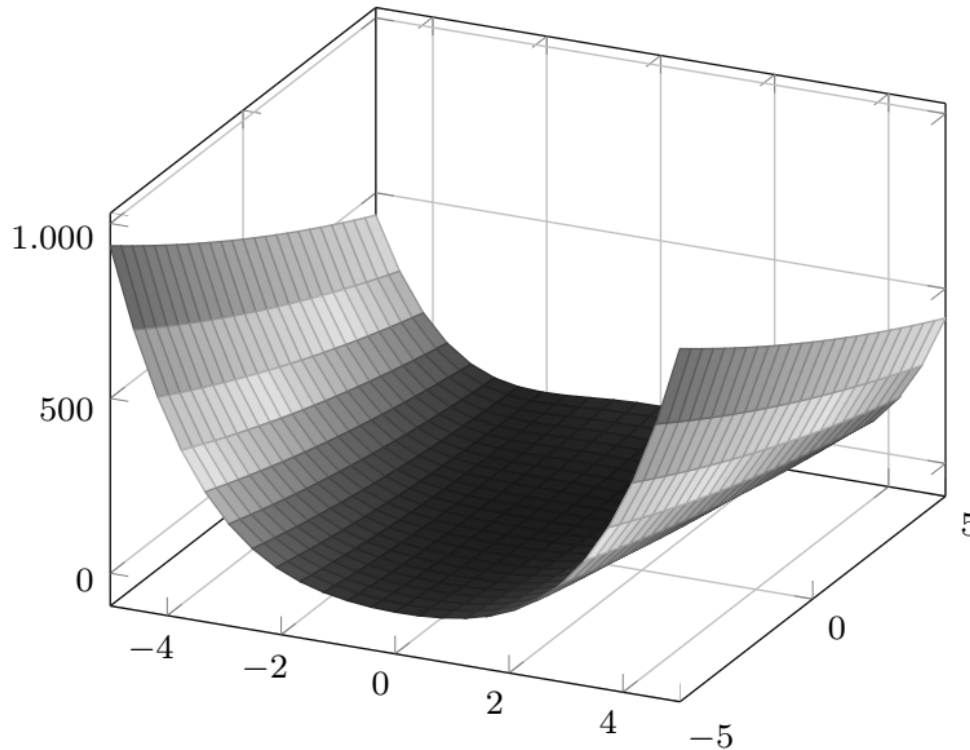
A fundamental result due to Bottou and Bousquet (2011) states that stochastic optimization algorithms (e.g., SGD) yield the best generalization performance (in terms of excess error) despite being the worst optimization algorithms for minimizing the empirical risk.

That is, **for a fixed computational budget, stochastic optimization algorithms reach a lower test error than more sophisticated algorithms** (2nd order methods, line search algorithms, etc) that would fit the training error too well or would consume too large a part of the computational budget at every step.



mnist

Momentum



In the situation of small but consistent gradients, as through valley floors, gradient descent moves **very slowly**.

SGD with Momentum



0:00 / 0:46

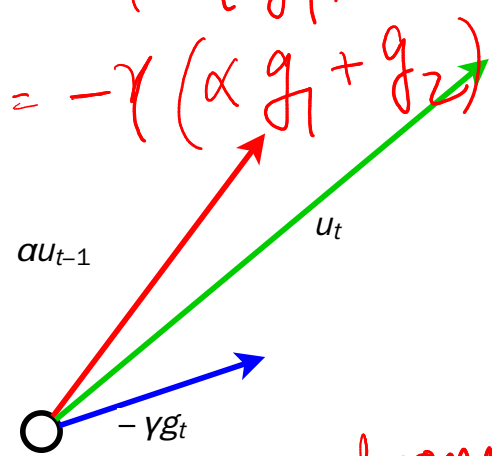


An improvement to gradient descent is to use **momentum** to add inertia in the choice of the step direction, that is

Handwritten notes and formulas:

- $u_3 \rightarrow \gamma(\alpha^2 g_1 + \alpha g_2 + g_3)$
- $u_t = \alpha u_{t-1} - \gamma g_t$
- $\theta_{t+1} = \theta_t + u_t$
- $u_0 = 0$
- $u_1 = -\gamma g_1$
- $u_2 = \alpha(-\gamma g_1) - \gamma g_2 = -\gamma(\alpha g_1 + g_2)$
- $u_t \rightarrow \gamma \sum_{i=0}^{t-1} \alpha^i g_i$

- The new variable u_t is the velocity. It corresponds to the direction and speed by which the parameters move as the learning dynamics progresses, modeled as an exponentially decaying moving average of negative gradients.
- Gradient descent with momentum has three nice properties:
 - it can go through local barriers,
 - it accelerates if the gradient does not change much,
 - it dampens oscillations in narrow valleys.



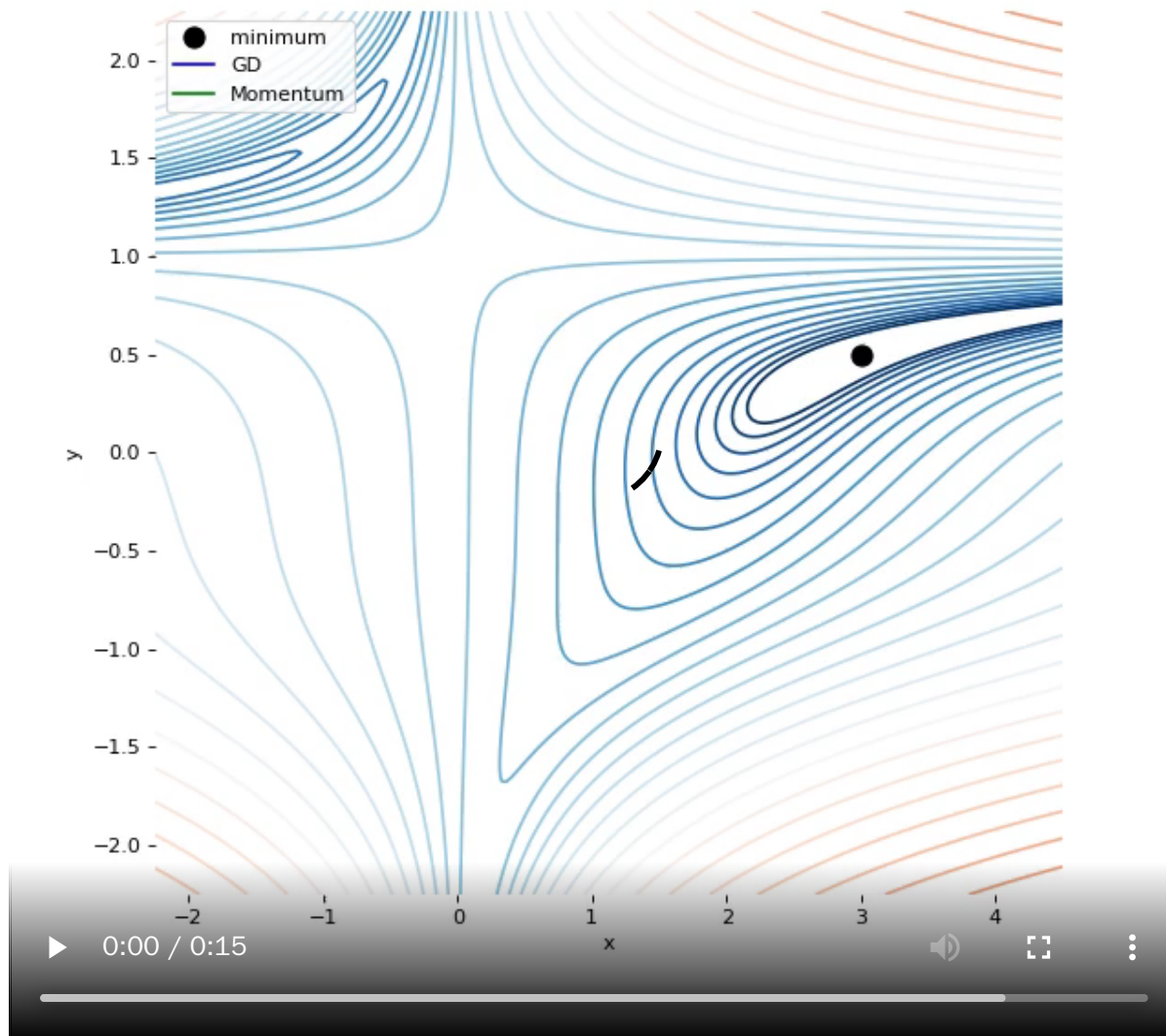
* In the above formula, I have taken g_t as the gradient for calculating the weights at t -th iteration. It is same as calculating the gradient considering the parameters from the $(t-1)$ th iteration.

~~If you are getting confused,~~
then you can replace g_t by g_{t-1} in the formula. It
is just a notation difference. If you replace
then g_{t-1} is the gradient considering the
parameters at $(t-1)$ th iteration.
The hyper-parameter α controls how recent gradients affect the current update.

- Usually, $\alpha = 0.9$, with $\alpha > \gamma$.
- If at each update we observed g , the step would (eventually) be

$$u = -\frac{\gamma}{1 - \alpha} g.$$

- Therefore, for $\alpha = 0.9$, it is like multiplying the maximum speed by 10 relative to the current direction.

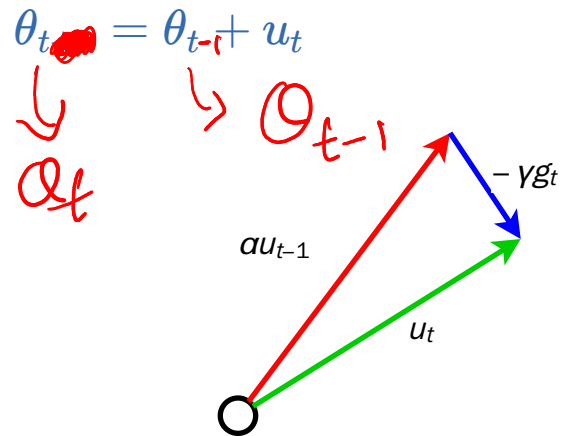


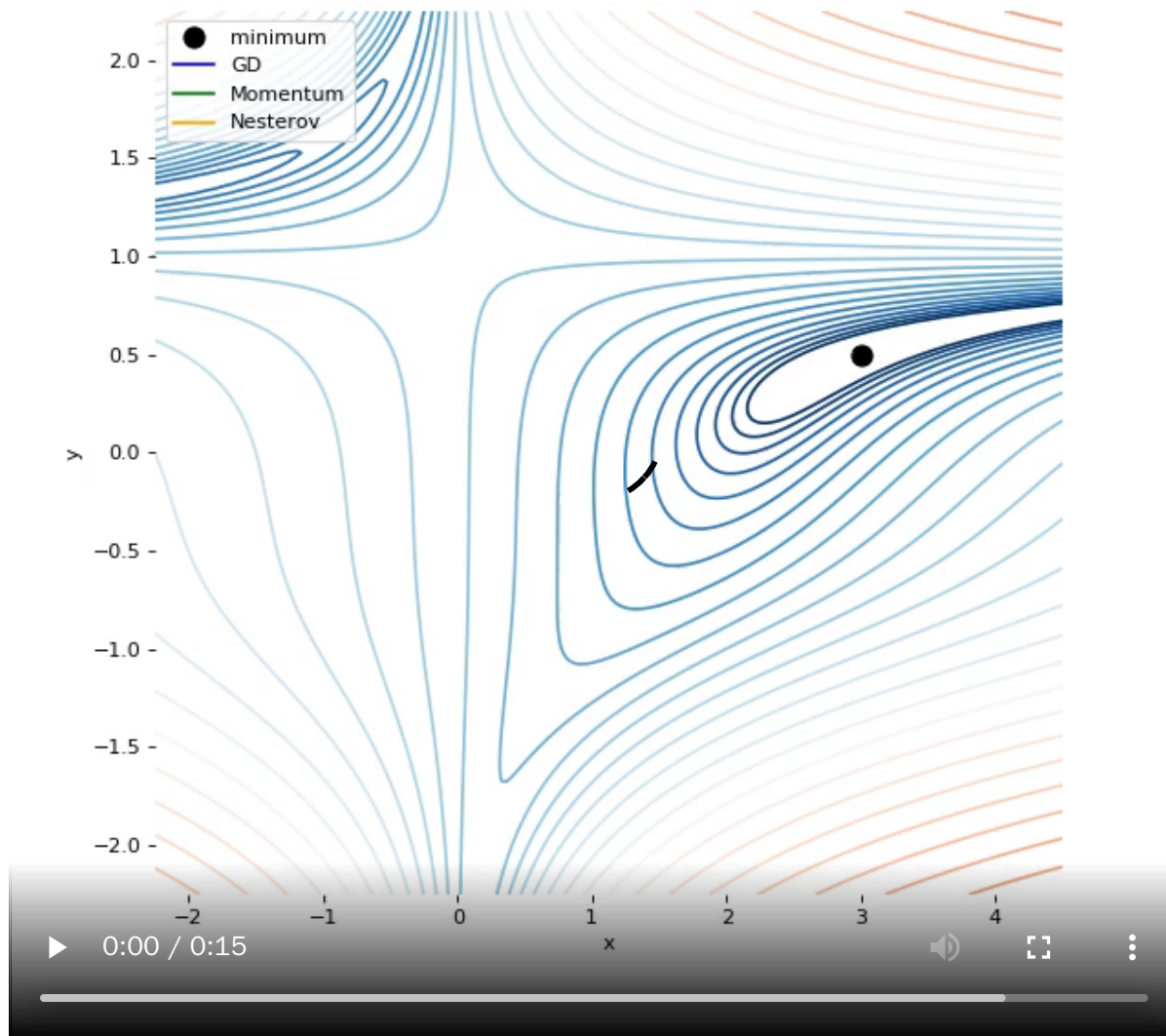
Nesterov momentum

An alternative consists in simulating a step in the direction of the velocity, then calculate the gradient and make a correction.

$$g_t = \frac{1}{N} \sum_{n=1}^N \nabla_{\theta} \ell(y_n, f(\mathbf{x}_n; \theta_t + \alpha u_{t-1}))$$

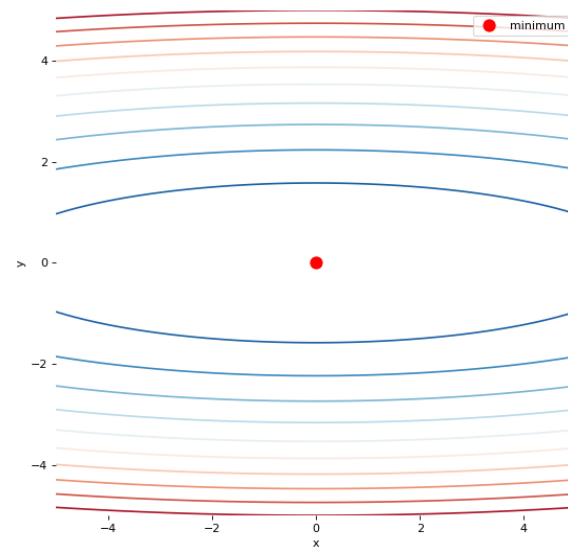
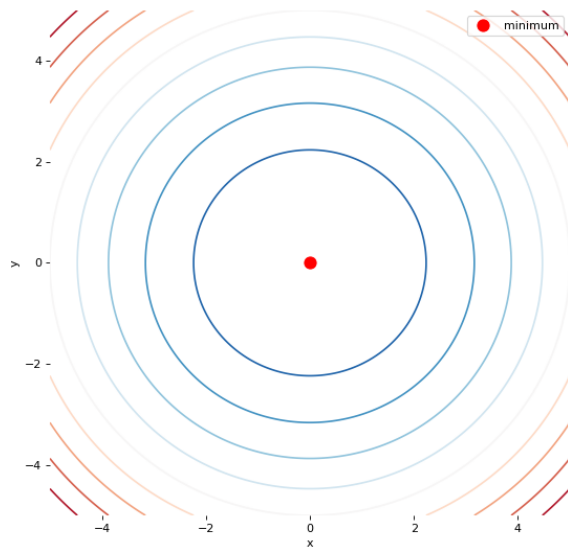
$$u_t = \alpha u_{t-1} - \gamma g_t$$





Adaptive learning rate

Vanilla gradient descent assumes the isotropy of the curvature, so that the same step size γ applies to all parameters.



Isotropic vs. Anisotropic

AdaGrad

Per-parameter downscale by square-root of sum of squares of all its historical values.

Size of r -vectors is the no. of params

$$r_t = r_{t-1} + g_t \odot g_t$$

$$\theta_t = \theta_{t-1} - \frac{\gamma}{\delta + \sqrt{r_t}} \odot g_t$$

Point-wise multiplication

$$r_0 = 0$$

if one param for multiple (P) parameters

- AdaGrad eliminates the need to manually tune the learning rate. Most implementation use $\gamma = 0.01$ as default.
- It is good when the objective is convex.
- r_t grows unboundedly during training, which may cause the step size to shrink and eventually become infinitesimally small.

$$r_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \end{bmatrix}_{p \text{ dim}}$$

* δ is a very small value (e.g. 10^{-8}), generally used to prevent the division by 0 in the initial stage when $r=0$.

RMSProp

Same as AdaGrad but accumulate an exponentially decaying average of the gradient.

$$r_t = \rho r_{t-1} + (1 - \rho) g_t \odot g_t$$
$$\theta_t = \theta_{t-1} - \frac{\gamma}{\delta + \sqrt{r_t}} \odot g_t.$$

(Handwritten red annotations: an arrow points from θ_t to θ_t , and another arrow points from θ_{t-1} to θ_{t-1})

- Perform better in non-convex settings.
- Does not grow unboundedly.

Adam

Similar to RMSProp with momentum, but with bias correction terms for the first and second moments.

same as
velocity in
momentum
optimization

$$s_t = \rho_1 s_{t-1} + (1 - \rho_1) g_t$$

$$\hat{s}_t = \frac{s_t}{1 - \rho_1^t}$$

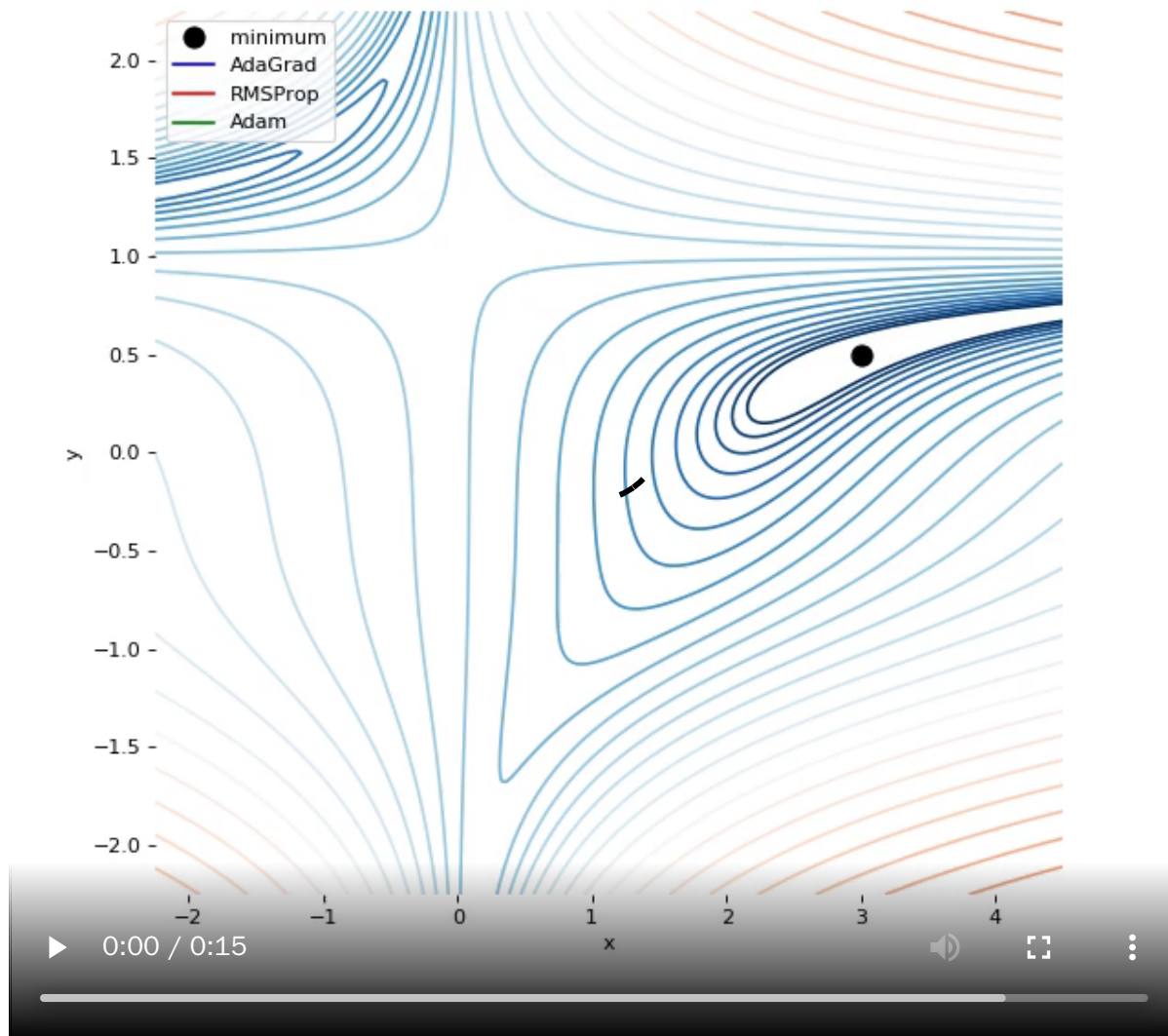
$$r_t = \rho_2 r_{t-1} + (1 - \rho_2) g_t \odot g_t$$

$$\hat{r}_t = \frac{r_t}{1 - \rho_2^t}$$

$$\theta_t = \theta_{t-1} - \gamma \frac{\hat{s}_t}{\delta + \sqrt{\hat{r}_t}}$$

same as
RMSProp

- Good defaults are $\rho_1 = 0.9$ and $\rho_2 = 0.999$.
- Adam is one of the **default optimizers** in deep learning, along with SGD with momentum.

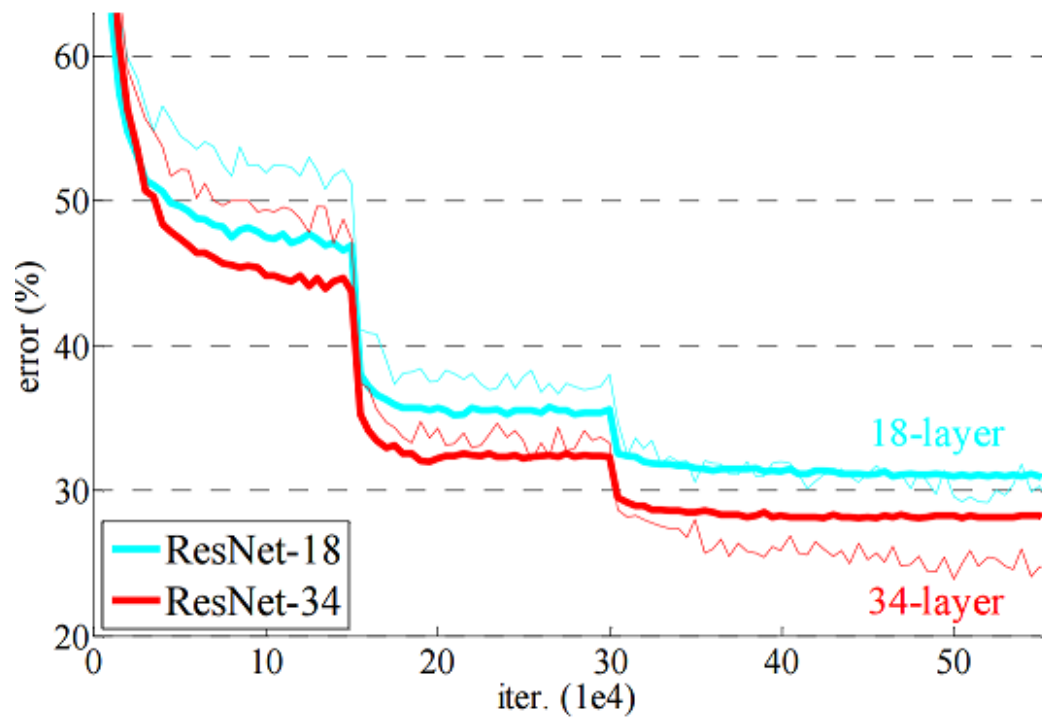


(demo)

Scheduling

Despite per-parameter adaptive learning rate methods, it is usually helpful to **anneal the learning rate** γ over time.

- Step decay: reduce the learning rate by some factor every few epochs (e.g, by half every 10 epochs).
- Exponential decay: $\gamma_t = \gamma_0 \exp(-kt)$ where γ_0 and k are hyper-parameters.
- $1/t$ decay: $\gamma_t = \gamma_0 / (1 + kt)$ where γ_0 and k are hyper-parameters.



Step decay scheduling for training ResNets.

Q1

Calculate the total number of trainable parameters for an MLP with the following architecture:

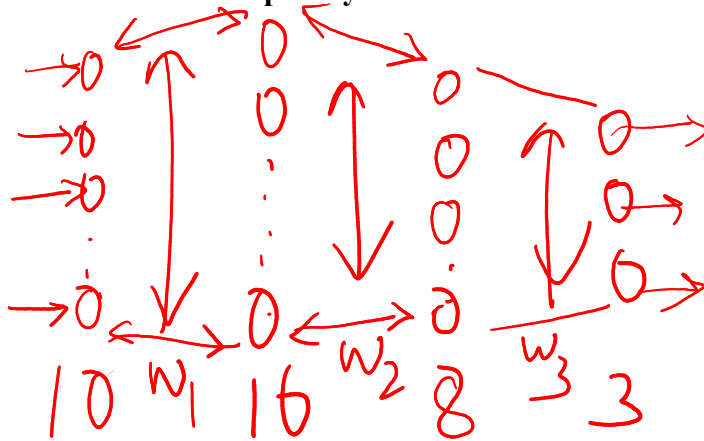
Input Layer: 10 neurons

Hidden Layer 1: 16 neurons

Hidden Layer 2: 8 neurons

Output Layer: 3 neurons (e.g., for a 3-class classification)

Assume every neuron in the hidden and output layers has a bias term.



$$w_1: 10 \times 16 + 16 = 176$$

$$w_2: 16 \times 8 + 8 = 136$$

$$w_3: 8 \times 3 + 3 = 27$$

$$\underline{339}$$

Q2

A classifier is designed to predict one of three classes: A, B, or C.

Suppose for a given data point: The true label is B.

The predicted probabilities by the model are:

$P(A) = 0.2$, $P(B) = 0.5$, $P(C) = ?$

Tasks:

Represent the true label using one-hot encoding.

Compute the cross-entropy loss for this prediction.

$$\begin{bmatrix} 0 & 1 & 0 \end{bmatrix}$$
$$\begin{bmatrix} 0.2 & 0.5 & 0.3 \end{bmatrix}$$

$$l_i = - \sum_{k=1}^3 y_{ik} \log(\hat{y}_{ik})$$
$$= - \log(0.5)$$

Q3

Given $\mathbf{w}_0 = 2\hat{i} + 3\hat{j}$, $\mathbf{u}_0 = 0$, $\eta = 0.1$, $\alpha = 0.9$, $\mathbf{g}_1 = 4\hat{i} + 2\hat{j}$, $\mathbf{g}_2 = \hat{i} + 3\hat{j}$, compute $\mathbf{u}_1$, $\mathbf{w}_1$ and $\mathbf{u}_2$, $\mathbf{w}_2$ using SGD with Momentum.

$$\mathbf{u}_t = \alpha \mathbf{u}_{t-1} - \eta \mathbf{g}_t$$

$$\mathbf{Q}_{t+1} = \mathbf{Q}_t + \mathbf{u}_t \quad \mathbf{u}_0 = 0$$

$$\begin{aligned} \mathbf{u}_1 &= 0.9 \times 0 - 0.1 (4\hat{i} + 2\hat{j}) \\ &= -0.4\hat{i} - 0.2\hat{j} \end{aligned}$$

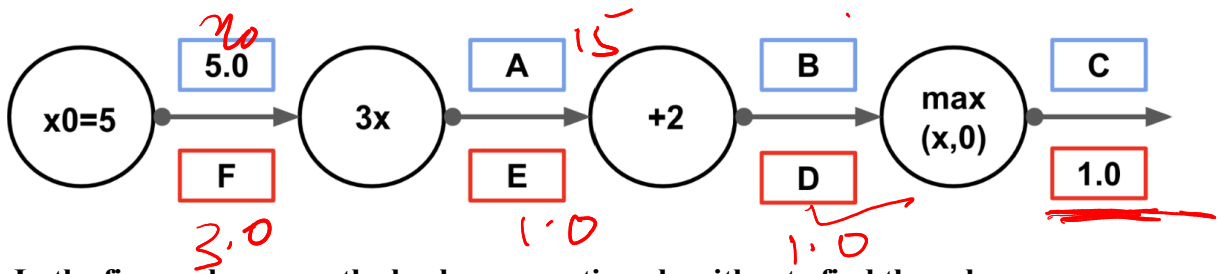
$$\begin{aligned} \mathbf{w}_1 &= \mathbf{w}_0 + \mathbf{u}_1 = (2\hat{i} + 3\hat{j}) - (0.4\hat{i} + 0.2\hat{j}) \\ &= \underline{1.6\hat{i} + 2.8\hat{j}} \end{aligned}$$

$$\begin{aligned} \mathbf{u}_2 &= 0.9(-0.4\hat{i} - 0.2\hat{j}) \\ &\quad - 0.1(\hat{i} + 3\hat{j}) \\ &= -0.46\hat{i} - 0.48\hat{j} \end{aligned}$$

$$\begin{aligned} \mathbf{w}_2 &= \mathbf{w}_1 + \mathbf{u}_2 = (1.6\hat{i} + 2.8\hat{j}) \\ &\quad - (0.46\hat{i} + 0.48\hat{j}) \end{aligned}$$

$$= \underline{1.14\hat{i} + 2.32\hat{j}}$$

Q4



In the figure above, use the back-propagation algorithm to find the values of A, B, C, D, E and F.

The upper blue boxes represent the forward pass values, and the lower red boxes represent the backward pass gradient value.

$$\frac{\partial l}{\partial C} = 1.0$$

$$D = \frac{\partial l}{\partial B} = \frac{\partial l}{\partial C} \cdot \frac{\partial C}{\partial B} = 1.0 \times 1.0 = \underline{1.0}$$

$$A = 3x_0 = 15$$

$$B = A + 2 = 17$$

$$C = \max(B, 0) = 17$$

$$E = \frac{\partial l}{\partial A} = \frac{\partial l}{\partial B} \cdot \frac{\partial B}{\partial A} = 1.0 \times 1.0 = \underline{1.0}$$

$$F = \frac{\partial l}{\partial x_0} = \frac{\partial l}{\partial A} \cdot \frac{\partial A}{\partial x_0} = 1.0 \times 3 = \underline{3.0}$$

Q5

Given $\theta_0 = (2, 3)$, $r_0 = (0, 0)$, $\rho = 0.9$, $\gamma = 0.1$, $\delta = 10^{-6}$, $g_1 = (4, 2)$, $g_2 = (1, 3)$, compute r_1, θ_1 and r_2, θ_2 using RMSProp.

$$\checkmark \quad \sigma_t = \rho \sigma_{t-1} + (1-\rho) g_t \odot g_t$$

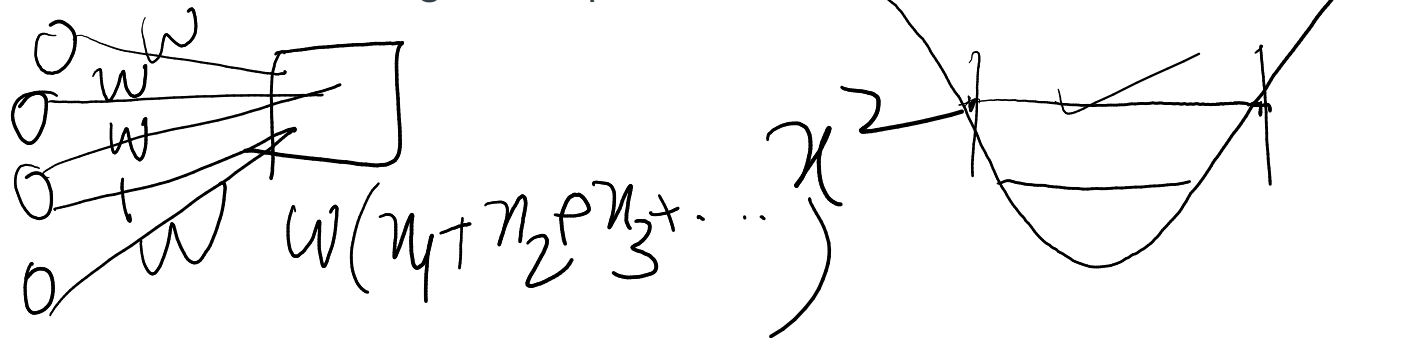
$$\theta_t = \theta_{t-1} - \frac{\gamma}{\delta + \sqrt{\sigma_t}} \odot g_t$$

Initialization

In convex problems, provided a good learning rate γ , convergence is guaranteed regardless of the initial parameter values.

In the non-convex regime, initialization is **much more important**! Little is known on the mathematics of initialization strategies of neural networks.

- What is known: initialization should break symmetry.
- What is known: the scale of weights is important.



Controlling for the variance in the forward pass

A first strategy is to initialize the network parameters such that activations preserve the **same variance across layers**.

Intuitively, this ensures that the information keeps flowing during the **forward pass**, without reducing or magnifying the magnitude of input signals exponentially.

Let us assume that

- we are in a linear regime at initialization (e.g., the positive part of a ReLU or the middle of a sigmoid),
- weights w_{ij}^l are initialized i.i.d,
- biases b_l are initialized to be 0,
- input features are i.i.d, with a variance denoted as $\mathbb{V}[x]$.

Then, the variance of the activation h_i^l of unit i in layer l is

$$\begin{aligned}\mathbb{V}[h_i^l] &= \mathbb{V}\left[\sum_{j=0}^{q_{l-1}-1} w_{ij}^l h_j^{l-1}\right] \\ &= \sum_{j=0}^{q_{l-1}-1} \mathbb{V}[w_{ij}^l] \mathbb{V}[h_j^{l-1}]\end{aligned}$$

where q_l is the width of layer l and $h_j^0 = x_j$ for all $j = 0, \dots, p - 1$.

Since the weights w_{ij}^l at layer l share the same variance $\mathbb{V}[w^l]$ and the variance of the activations in the previous layer are the same, we can drop the indices and write

$$\mathbb{V}[h^l] = q_{l-1} \mathbb{V}[w^l] \mathbb{V}[h^{l-1}].$$

Therefore, the variance of the activations is preserved across layers when

$$\mathbb{V}[w^l] = \frac{1}{q_{l-1}} \quad \forall l.$$

This condition is enforced in **LeCun's uniform initialization**, which is defined as

$$w_{ij}^l \sim \mathcal{U} \left[-\sqrt{\frac{3}{q_{l-1}}}, \sqrt{\frac{3}{q_{l-1}}} \right].$$

Controlling for the variance in the backward pass

A similar idea can be applied to ensure that the gradients flow in the **backward pass** (without vanishing nor exploding), by maintaining the variance of the gradient with respect to the activations fixed across layers.

Under the same assumptions as before,

$$\begin{aligned}\mathbb{V} \left[\frac{d\hat{y}}{dh_i^l} \right] &= \mathbb{V} \left[\sum_{j=0}^{q_{l+1}-1} \frac{d\hat{y}}{dh_j^{l+1}} \frac{\partial h_j^{l+1}}{\partial h_i^l} \right] \\ &= \mathbb{V} \left[\sum_{j=0}^{q_{l+1}-1} \frac{d\hat{y}}{dh_j^{l+1}} w_{j,i}^{l+1} \right] \\ &= \sum_{j=0}^{q_{l+1}-1} \mathbb{V} \left[\frac{d\hat{y}}{dh_j^{l+1}} \right] \mathbb{V} [w_{ji}^{l+1}]\end{aligned}$$

If we further assume that

- the gradients of the activations at layer l share the same variance
- the weights at layer $l + 1$ share the same variance $\mathbb{V} [w^{l+1}]$,

then we can drop the indices and write

$$\mathbb{V} \left[\frac{d\hat{y}}{dh^l} \right] = q_{l+1} \mathbb{V} \left[\frac{d\hat{y}}{dh^{l+1}} \right] \mathbb{V} [w^{l+1}] .$$

Therefore, the variance of the gradients with respect to the activations is preserved across layers when

$$\mathbb{V} [w^l] = \frac{1}{q_l} \quad \forall l.$$

Xavier initialization

We have derived two different conditions on the variance of w^l ,

- $\mathbb{V}[w^l] = \frac{1}{q_{l-1}}$
- $\mathbb{V}[w^l] = \frac{1}{q_l}$.

A compromise is the **Xavier initialization**, which initializes w^l randomly from a distribution with variance

$$\mathbb{V}[w^l] = \frac{1}{\frac{q_{l-1} + q_l}{2}} = \frac{2}{q_{l-1} + q_l}.$$

For example, **normalized initialization** is defined as

$$w_{ij}^l \sim \mathcal{U} \left[-\sqrt{\frac{6}{q_{l-1} + q_l}}, \sqrt{\frac{6}{q_{l-1} + q_l}} \right].$$

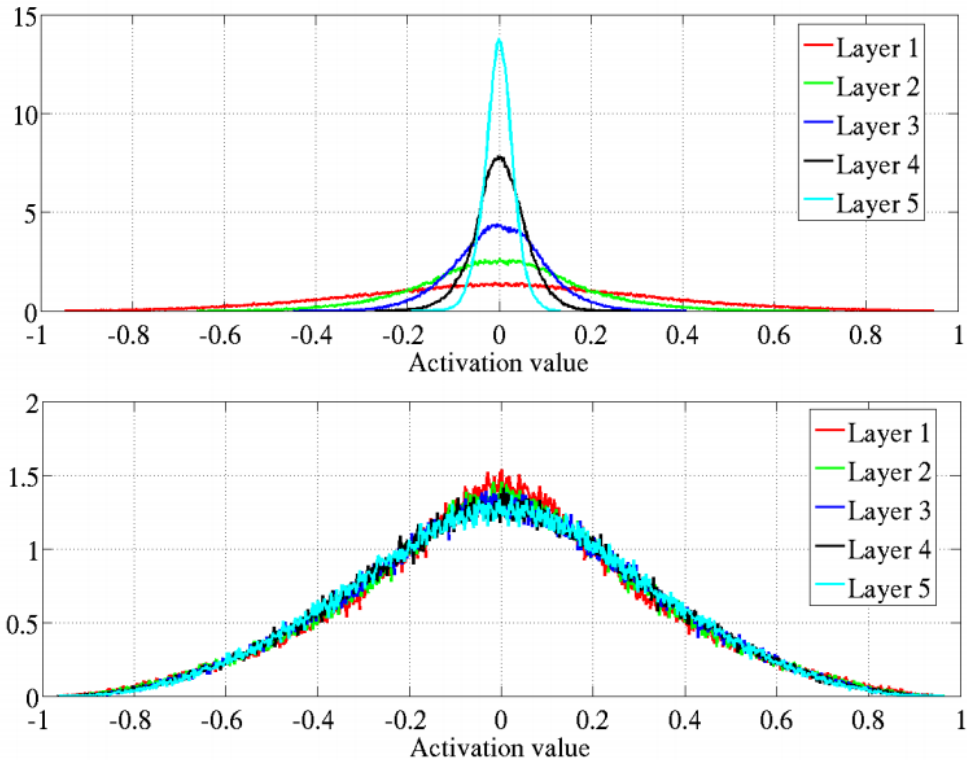


Figure 6: *Activation values normalized histograms with hyperbolic tangent activation, with standard (top) vs normalized initialization (bottom). Top: 0-peak increases for higher layers.*

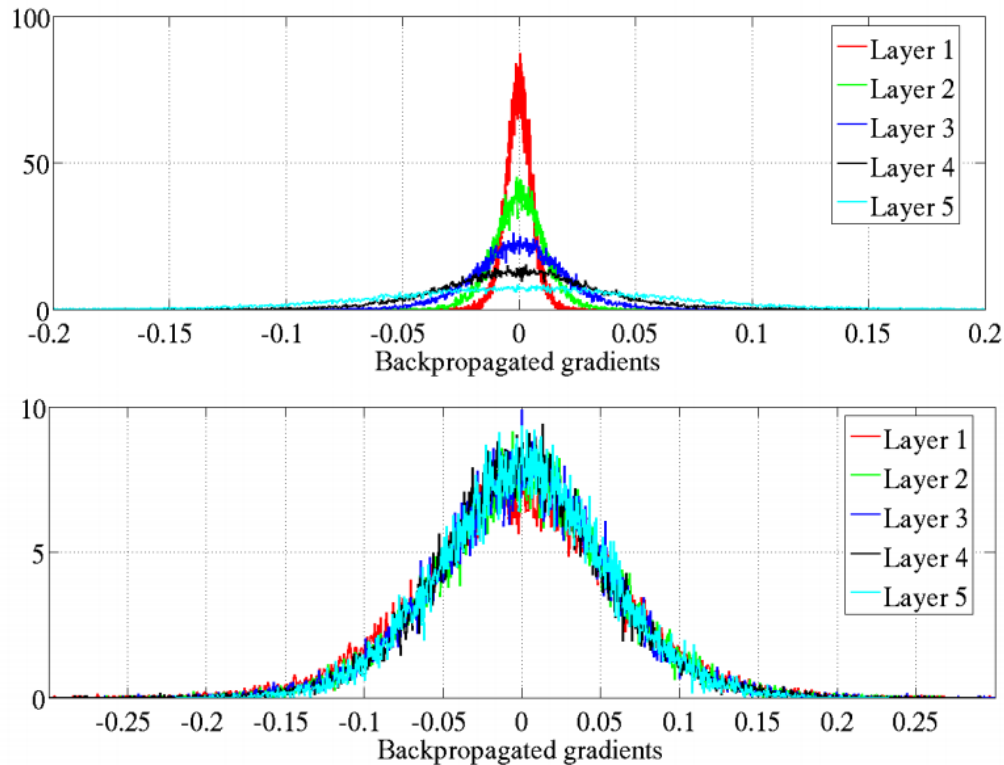


Figure 7: *Back-propagated gradients normalized histograms with hyperbolic tangent activation, with standard (top) vs normalized (bottom) initialization. Top: 0-peak decreases for higher layers.*

Normalization

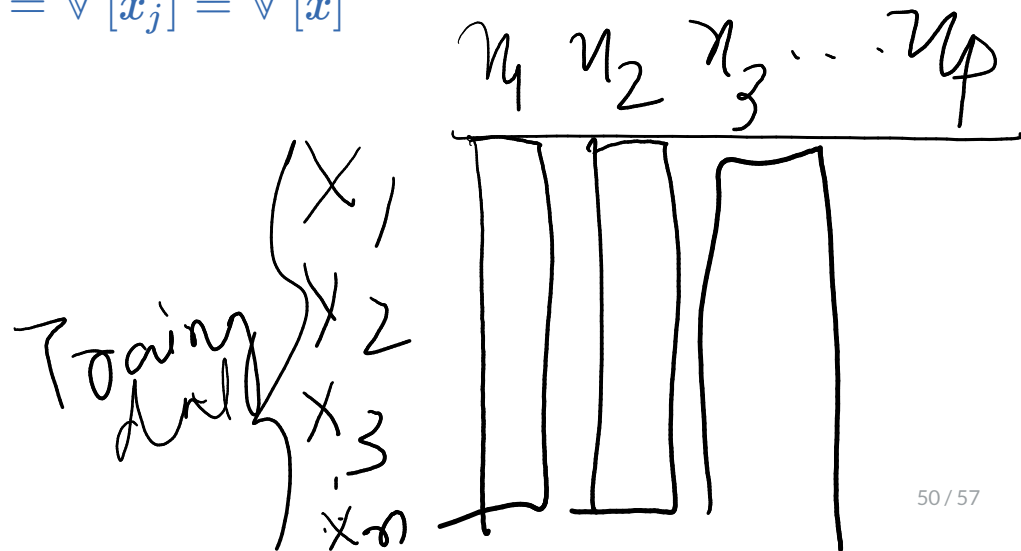
Data normalization

Previous weight initialization strategies rely on preserving the activation variance constant across layers, under the initial assumption that the **input feature variances are the same**.

That is,

$$\mathbb{V}[x_i] = \mathbb{V}[x_j] \triangleq \mathbb{V}[x]$$

for all pairs of features i, j .



In general, this constraint is not satisfied but can be enforced by **standardizing** the input data feature-wise,

$a_1, a_2, \dots, a_n$

where

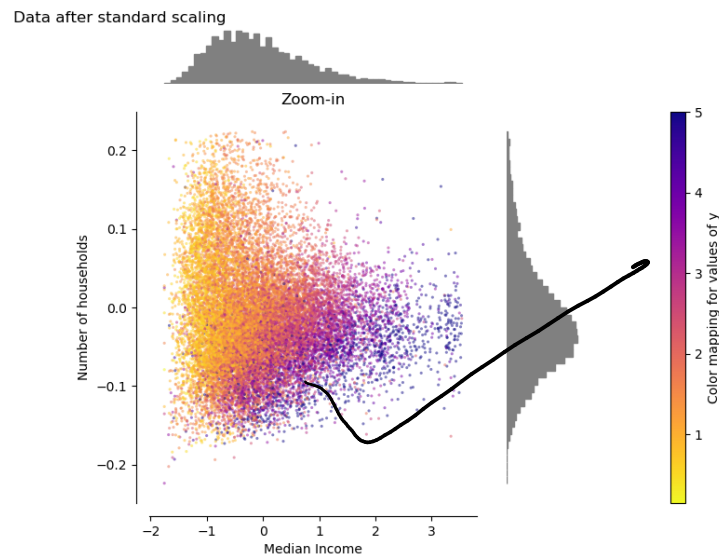
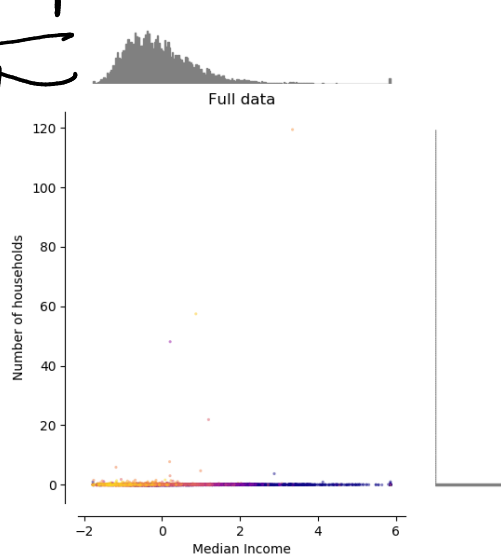
$$\hat{\mu} = \frac{1}{N} \sum_{\mathbf{x} \in \mathbf{d}} \mathbf{x}$$

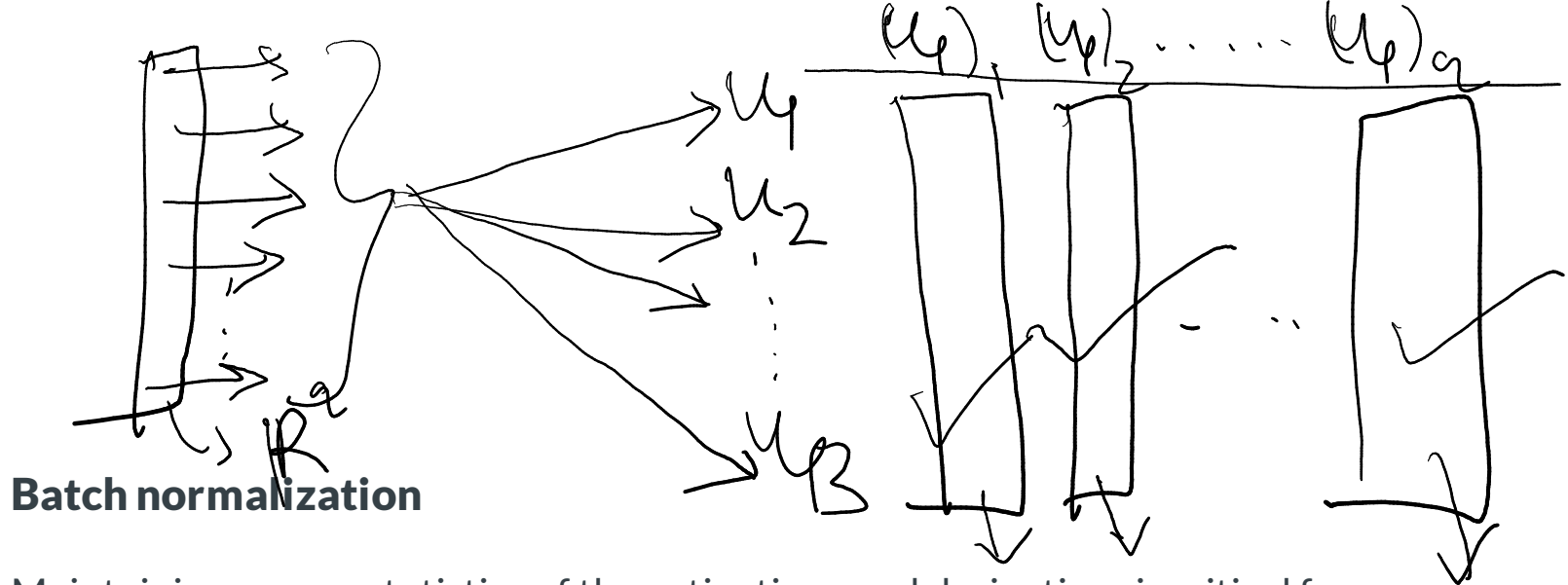
$$\mathbf{x}' = (\mathbf{x} - \hat{\mu}) \odot \frac{1}{\hat{\sigma}},$$

$$\frac{x - \mu}{\sigma}$$

$$\hat{\sigma}^2 = \frac{1}{N} \sum_{\mathbf{x} \in \mathbf{d}} (\mathbf{x} - \hat{\mu})^2.$$

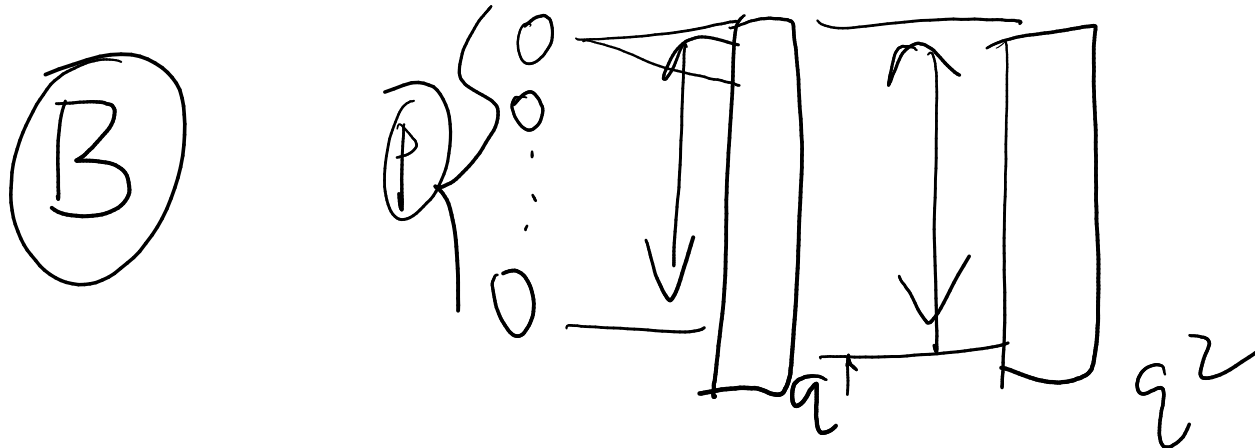
$$x'_i = \frac{x_i - \mu}{\sigma}$$





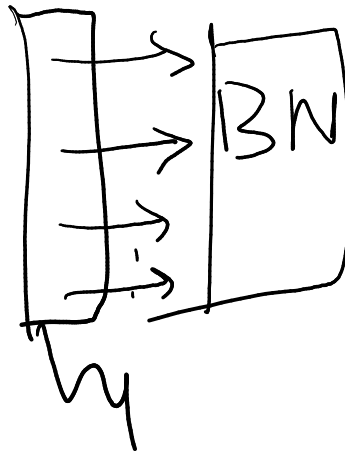
Maintaining proper statistics of the activations and derivatives is critical for training neural networks.

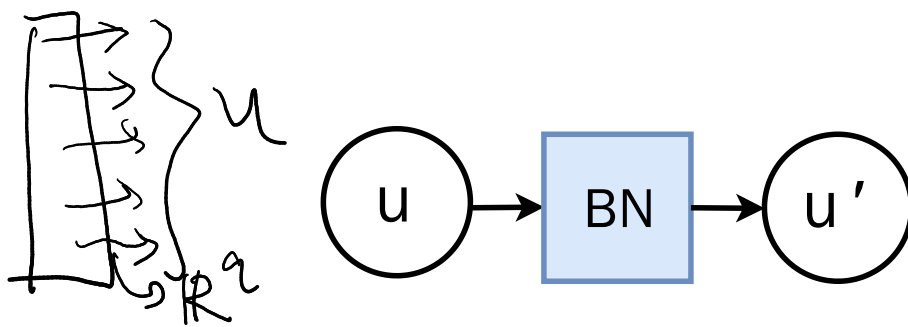
This constraint can be enforced explicitly during the forward pass by re-normalizing them. **Batch normalization** was the first method introducing this idea.



During training, batch normalization shifts and rescales according to the mean and variance estimated on the batch.

During test, it shifts and rescales according to the empirical moments estimated during training.





Let us consider a minibatch of samples at training, for which $\mathbf{u}_b \in \mathbb{R}^q$, $b = 1, \dots, B$, are intermediate values computed at some location in the computational graph.

In batch normalization following the node $\mathbf{u}$, the per-component mean and variance are first computed on the batch

$$\hat{\mu}_{\text{batch}} = \frac{1}{B} \sum_{b=1}^B \mathbf{u}_b \quad \hat{\sigma}_{\text{batch}}^2 = \frac{1}{B} \sum_{b=1}^B (\mathbf{u}_b - \hat{\mu}_{\text{batch}})^2,$$

from which the standardized $\mathbf{u}'_b \in \mathbb{R}^q$ are computed such that

$$\mathbf{u}'_b = \gamma \odot (\mathbf{u}_b - \hat{\mu}_{\text{batch}}) \odot \frac{1}{\hat{\sigma}_{\text{batch}} + \epsilon} + \beta \rightarrow \gamma \odot \mathbf{u}'_b + \beta$$

where $\gamma, \beta \in \mathbb{R}^q$ are parameters to optimize. They allow the network to recover any required distribution if normalization becomes too restrictive.

Exercise: How does batch normalization combine with backpropagation?

During inference, batch normalization shifts and rescales each component according to the empirical moments estimated during training:

$$\mathbf{u}' = \gamma \odot (\mathbf{u} - \hat{\mu}) \odot \frac{1}{\hat{\sigma}} + \beta.$$

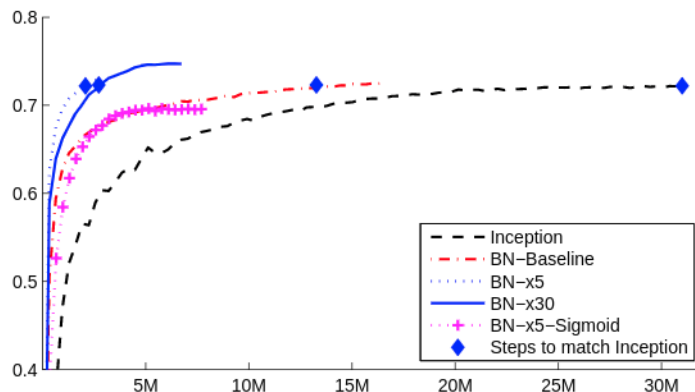


Figure 2: Single crop validation accuracy of Inception and its batch-normalized variants, vs. the number of training steps.

Model	Steps to 72.2%	Max accuracy
Inception	$31.0 \cdot 10^6$	72.2%
<i>BN-Baseline</i>	$13.3 \cdot 10^6$	72.7%
<i>BN-x5</i>	$2.1 \cdot 10^6$	73.0%
<i>BN-x30</i>	$2.7 \cdot 10^6$	74.8%
<i>BN-x5-Sigmoid</i>		69.8%

Figure 3: For Inception and the batch-normalized variants, the number of training steps required to reach the maximum accuracy of Inception (72.2%), and the maximum accuracy achieved by the network.

Problem: Batch Normalization with Two Features

Consider a mini-batch of 3 samples
each with two activated outputs in some hidden layer of a neural network

$$u = \begin{bmatrix} 4 & 8 \\ 2 & 6 \\ 6 & 13 \end{bmatrix}$$

Given parameters:

- γ (gamma) = $\begin{bmatrix} 2 & 3 \end{bmatrix}$
- β (beta) = $\begin{bmatrix} 1 & 2 \end{bmatrix}$
- ϵ (epsilon) = 10^{-5}

Compute the **normalized output** after batch normalization.

Solution in next page

Ans Mean $\mu = \begin{bmatrix} \mu_1 & \mu_2 \end{bmatrix} = \begin{bmatrix} \frac{4+2+6}{3}, \frac{8+6+10}{3} \end{bmatrix}$
 $= [4, 9]$

Standard deviations
 $\sigma_1^2 = \frac{1}{3} \{ (4-4)^2 + (2-4)^2 + (6-4)^2 \}$
 $= 2.67$

$\sigma_2^2 = \frac{1}{3} \{ (8-9)^2 + (6-9)^2 + (10-9)^2 \}$
 $= 8.67$

$\therefore \sigma = [\sigma_1 \quad \sigma_2] = [\sqrt{2.67} \quad \sqrt{8.67}]$
 $= [1.63 \quad 2.94]$

For normalized $\hat{u}$

$\hat{u}_i = \frac{u_i - \mu}{\sigma + \epsilon}$ if $\sigma = 0$, use ϵ

$\therefore \hat{u} = \begin{bmatrix} \frac{4-4}{1.63} & \frac{8-9}{2.94} \\ \frac{2-4}{1.63} & \frac{6-9}{2.94} \\ \frac{6-4}{1.63} & \frac{10-9}{2.94} \end{bmatrix} = \begin{bmatrix} 0 & -0.34 \\ -1.22 & -1.02 \\ 1.22 & 1.36 \end{bmatrix}$

Apply the scale and shift $\rightarrow$

$$V_i = \gamma \hat{u}_i + \beta \quad \gamma = \begin{bmatrix} 2 & 3 \end{bmatrix}$$

$\downarrow$
in scalar form

$$\beta = \begin{bmatrix} 1 & 2 \end{bmatrix}$$

$$\therefore V = \gamma \odot \hat{u} + \beta \quad \text{in matrix form}$$

$$\therefore V = \begin{bmatrix} 2 & 3 \end{bmatrix} \odot \begin{bmatrix} 0 & -0.34 \\ -1.22 & -1.02 \\ 1.22 & 1.36 \end{bmatrix} + \begin{bmatrix} 1 & 2 \end{bmatrix}$$

$$= \begin{bmatrix} 2 \times 0 + 1 & 3 \times -0.34 + 2 \\ 2 \times -1.22 + 1 & 3 \times -1.02 + 2 \\ 2 \times 1.22 + 1 & 3 \times 1.36 + 2 \end{bmatrix}$$

$$\approx \begin{bmatrix} 1 & 0.98 \\ -1.44 & -1.06 \\ 3.44 & 4.08 \end{bmatrix} \begin{matrix} \rightarrow \text{sample 1} \\ \rightarrow \text{sample 2} \\ \rightarrow \text{sample 3} \end{matrix}$$

is the normalized output.

*Credits & License: Adapted from materials by Prof. Gilles Louppe (ULiège).
Source: github.com/glouppe/info8010-deep-learning Licensed under BSD 3-Clause.*